SciLifeLab

How to use LLMs to present results from your research project as a web application

DDLS Annual Conference 2026, 2026-11-10

Arnold Kochari (he/him)
Project lead / Product owner
ORCID: https://orcid.org/0000-0003-1373-5121

Mahbub Ul Alam (he/him)
Data engineer (AI)
ORCID: https://orcid.org/0000-0002-1101-3793

Agenda

13:30	Start
13:35 - 13:50	Presenting results as web apps
13:50 - 14:15	Using LLM tools to create web apps
14:15 - 14:25	Publishing web apps on SciLifeLab Serve
14:25 - 14:35	Fika
14:35 - 15:15	Hands-on session
15:15 - 15:20	Funder and institutional requirements
15:20 - 15:30	Q&A
15:30 End	

Today we do not talk about doing analyses/creating a tool within a research project.

We will not give you a new miracle way of working with LLMs today.

SciLifeLab

Part 1: Presenting results as web apps

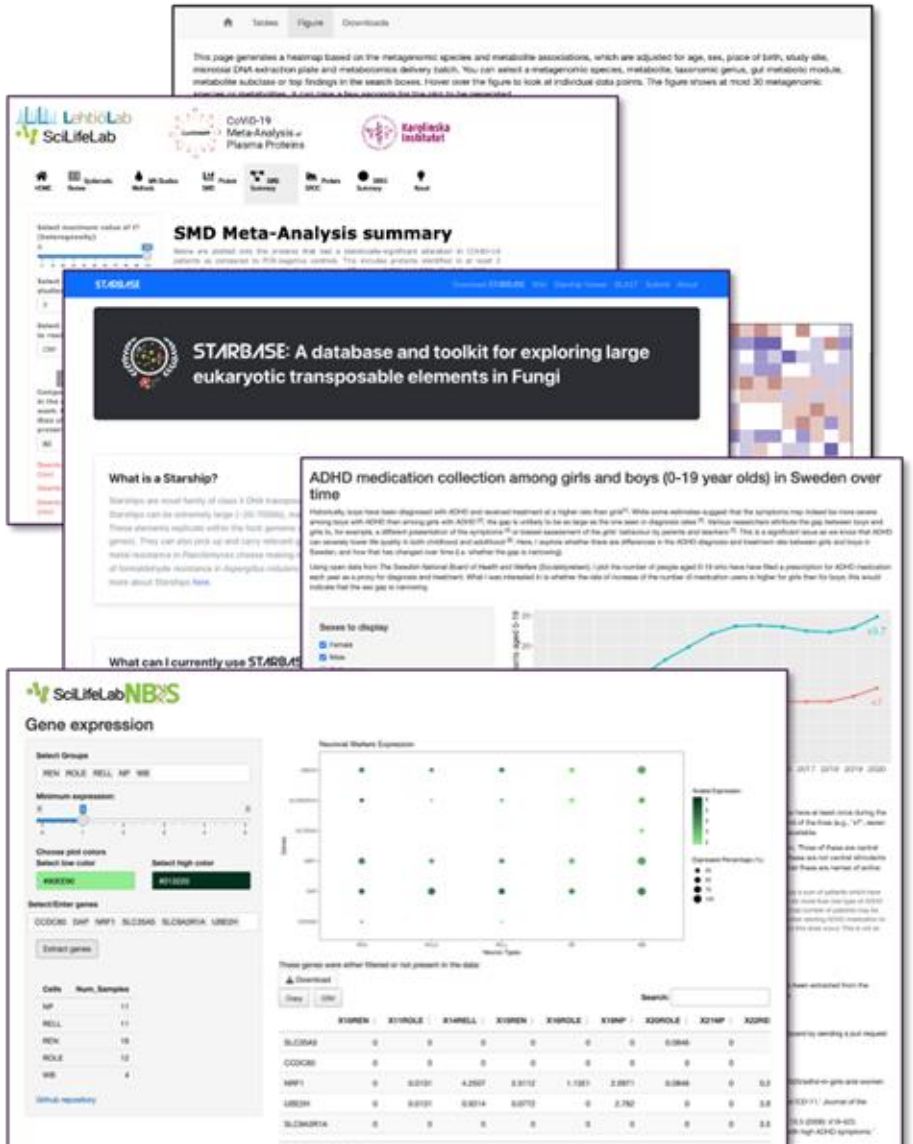
Outputs from a research project

- Traditional research outputs: journal article, conference paper, book chapter, dataset, analysis code.

- For results of a research study:
  - a static file is not always the best way to present results;
  - not every output can be **used by a reader independently**;
  - we may want to **continuously update** the output;
  - we may want to **let others contribute** to the output.
- Methods/tools development is a big part of some research groups' work (DDLS), these are also research results that need to be shared.

- Alternative ways to share results:
  - Web applications available as websites
  - Packages (Python, R)
  - Environments/containers (conda, Docker)
  - …

How web apps can be used in a research environment



- As **supplementary materials** for publications - **readers** can visualise and explore the data in detail
- As a quick and easy graphical user interface to allow **collaborators** to **explore or annotate data**
- To make **analysis tools** that allow users to upload their own data and interact with it
- Let others use your **machine learning models to make inference** (predictions) **on own data**
- As demo apps for a future **commercialisation effort**

Building web apps is not something we were taught to do



- Existing open source web frameworks make it possible.
  - Some frameworks especially for scientific data formats and typical use cases.
- Doable based on documentation
- Can be sped up with AI assistance tools

Frameworks for building web apps: R

- Open source framework for creating interactive web applications using R code:
  - power of packages for dealing with scientific formats or analyses
  - power of R for analysis and visualisation scripts

- Shiny apps can contain interactive visualisations, tables, text, images, take input files, etc.
- Easy to create for an R user/scripter:
  - no need for any knowledge of web development
  - lots of materials and help available online – e.g., https://mastering-shiny.org/ (CC BY-NC-ND 4.0 license)



Shiny
https://shiny.posit.co/r/

Frameworks for building web apps: Python



- Different focus and strengths in each case.
- Plotly Dash has been around for longer; Streamlit and Gradio “cool new kids on the block”.



Plotly Dash
https://dash.plotly.com/

- **Steeper learning curve but gives more flexibility.**
- Best for complex analytics applications with dependencies between filters, graphs, tables, and interactive components.



Streamlit
https://streamlit.io/

- Focus on simplicity.
- **Best for data dashboards, exploration of different projects/data types in one app.**
- Good for machine learning inference apps.

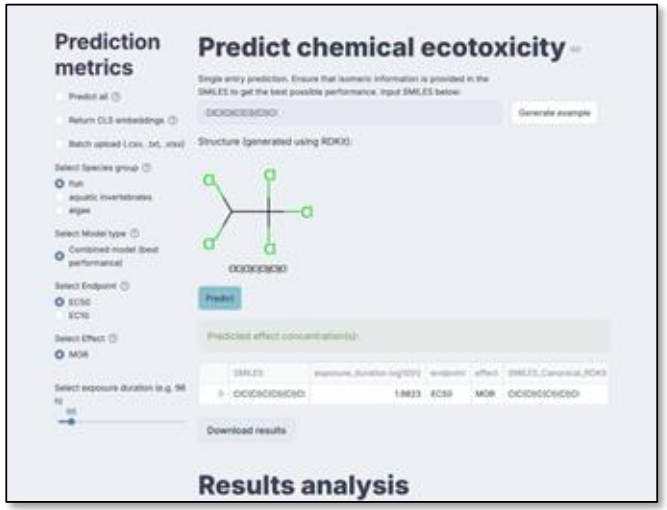


Gradio
https://gradio.app/

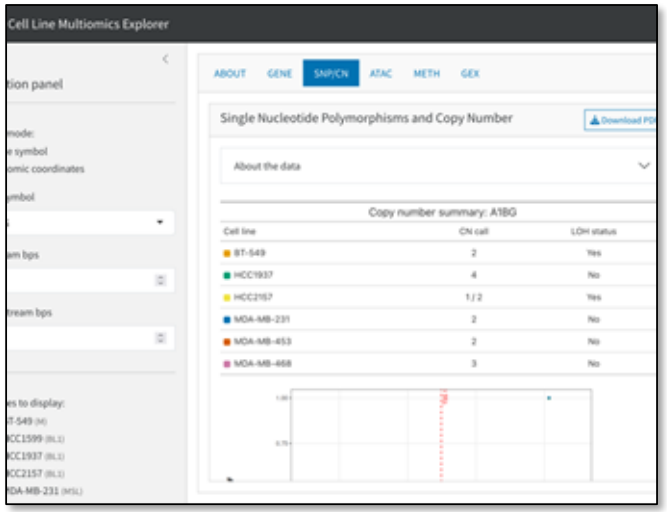
- Focus on simplicity.
- **Best for machine learning inference or tools with input (files) apps.**
- Built-in queuing for long-running tasks.

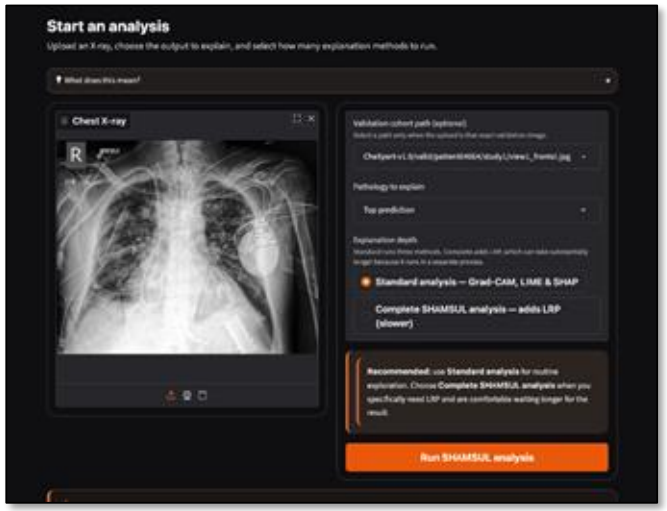
Examples

https://trident.serve.scilifelab.se/

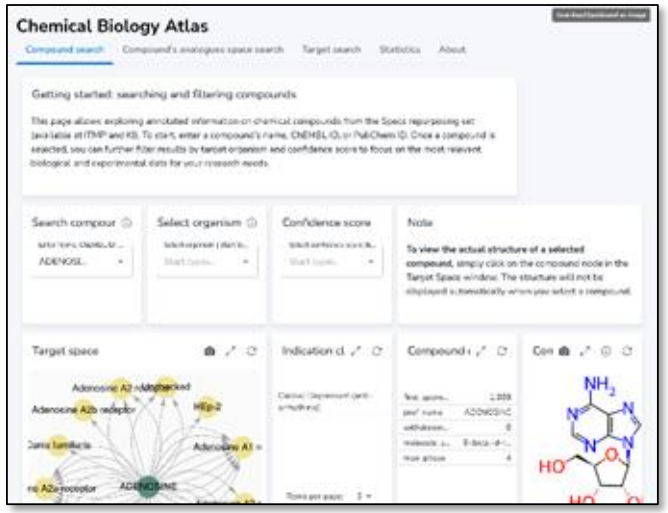


https://tnbc-cell-line-multiomics-explorer.serve.scilifelab.se





https://shamsul.serve.scilifelab.se/



https://chembioatlas.serve.scilifelab.se/

Not everything needs to be a web app

When it makes sense:

- Real scientific value added by the web app that you create and share.
- Non-standard file formats requiring special code to work with (e.g. special packages)
- Large datasets or heavy computations required

BUT: apps with/for sensitive data cannot be (easily) hosted anywhere so even if it would fulfil the criteria above, sometimes not a good idea to make these.



Source: Jim Pennucci, Flickr. License: CC BY 2.0.

Not everything needs to be a web app



Source: Arnold Kochari, taken at EMBL Heidelberg in March 2026

Web apps are easy to make but not easy to maintain:

- Available on a URL to everyone, but not easily portable.
- Requires suitable professional hosting.
- Updates will be required. Over time frameworks/packages need to be updated because there will be security vulnerabilities will be discovered.

SciLifeLab

Part 2: Using LLM tools to create web apps

Coding assistant, agent, harness



AI coding assistant

- Helps write, explain, review, and debug code.
- In a basic chat, you copy its code into files and run it yourself.

AI coding agent

- Can take coding actions with permission: read project files, edit code, run commands or tests, and respond to errors.
- You still approve or review what it changes.

Agent harness

- The software around the AI model that gives it tools, access to project files, instructions, and permission controls.
- The model generates responses and proposed actions; the harness manages the tool-based workflow.

Coding assistant, agent, harness

Assistant = helps

Agent = can act

Harness = tools and controls that enable acting

“Modern assistants can also combine all of these properties.”

Ways to work with AI

Browser, including free access (ChatGPT, Claude, Mistral, Kimi):

- Describe the app, request code, copy it into local files, and test it.
- Good starting point if you do not already have a coding tool.

In your code editor (Cursor, Windsurf, Continue.dev, Cline):

- Ask questions and generate or edit code while working in your project.
- Agent features may inspect files and carry out edits or commands, depending on mode and permissions.

Ways to work with AI

CLI with API key (OpenCode, Pi, Codex CLI, Qwen Code + SciLifeLab Open LLM):

- A terminal-based agent workflow for reading/editing a project and running development commands.

*CLI = Command-Line Interface

*API = Application Programming Interface

“Use whatever you already know. No specific AI product is required.”

What changes when we use an agent?

Prompt: “Build App with given specification”

With a chat-based assistant:

- It writes code and instructions.
- You save files, run the app, and paste errors back into chat.

With a coding agent and harness:

- It can inspect the project and create or change files.
- It may run the app or tests and use the error output to revise the code.
- You review what changed and verify that the scientific values are correct.

“Neither guarantees correct results.”

Two separate tasks: scientific implementation and web application



Task 1: Implementation and validation of your scientific results (with or without AI, outside the scope of today's workshop)

Task 2: Make your scientific results accessible through a web application (with AI).

- Use the validated scientific implementation as the core of the web application.
- Keep the web application code separate from the scientific logic.
- Build a user-friendly interface for providing inputs and presenting results.
- Verify that the interface correctly passes inputs to the underlying implementation and accurately displays its outputs.

How do we generally build a web app?

- Start with a research question and a dataset or analysis script.
- Decide who the app is for and what they should be able to do.
- Choose a simple framework and specify the app's outputs.
- Ask an AI assistant to build a small first version.
- Run it locally and examine what it does.
- Check the data, calculations, user interactions, and errors.
- When something fails: see the exact error and ask for the smallest fix.
- Improve one feature at a time.
- Iterate in small, testable steps

“Prompt → build → run → check → refine”

A good prompt is a small specification



- **Purpose:** What question should the app answer?
- **Data:** What does the input mean? What are its units?
- **Audience:** Who will use the app?
- **Interactions:** Filters, parameters, buttons, uploads, or downloads?
- **Output:** A chart/table, or a computed prediction/result?
- **Scientific rules:** What must not change or be invented?
- **Constraints:** Simple code, familiar framework, public/non-sensitive data.
- **Verification:** What checks should the app perform?

“Ask the assistant to state assumptions before rather than silently guessing.”

A working app is not necessarily a correct app



- **Research question**: What problem are we trying to solve?
- **Input:** What data will the application use? What do the variables represent?
- **Output:** What results should the application produce?
- **Scientific methods:** What calculations, algorithms, or models should be used, and why?
- **Assumptions and limitations:** What are the boundaries of the analysis?
- **Validation:** How will we know that the results are correct?

“Critically review your own app, improve it based on your observations, and ask a colleague to test it.”

A working app is not necessarily a correct app



- The researcher provides the scientific knowledge
  - makes methodological decisions
  - remains accountable for the results.
- AI can only help implement these decisions
- Keep the scientific code you trust. Ask AI to help build the interface around it.
- AI must not silently invent scientific methods or assumptions.

“You remain responsible for the scientific correctness of your application, even when AI writes the code.”

A working app is not necessarily a correct app



Check the software and security

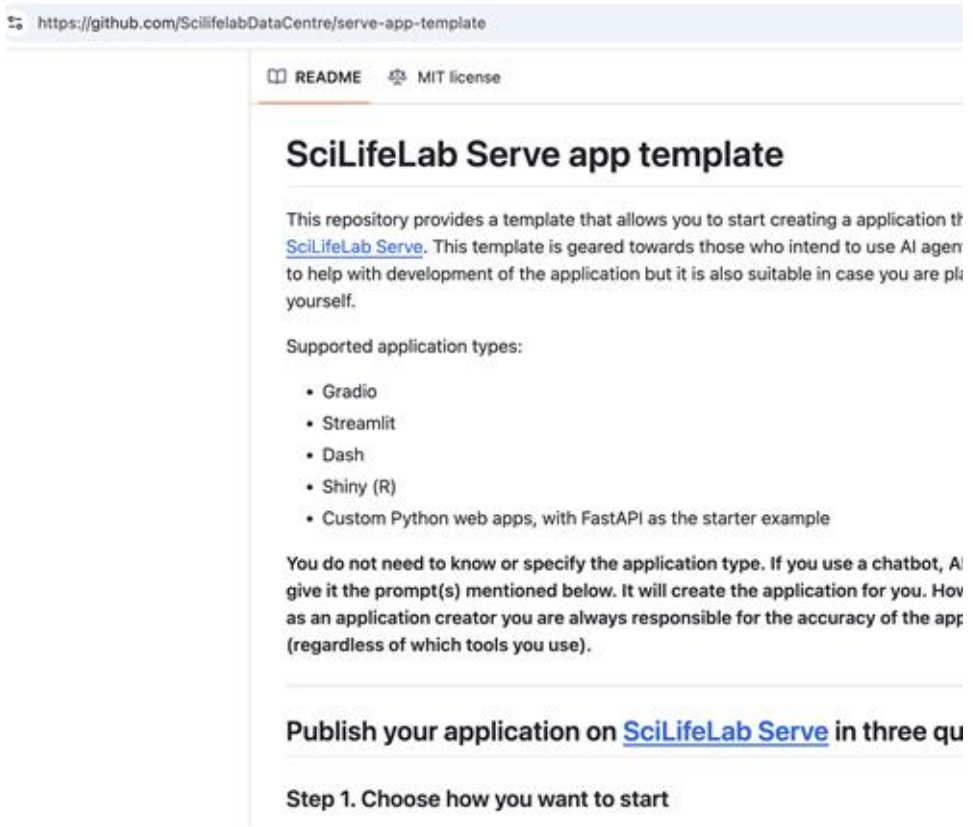
- Do filters and buttons work?
- Are dependencies, setup steps, and limitations documented?
- Have we excluded private data, tokens, and credentials?

“You remain responsible for interpretation, ethics, consequence, legality, and security.”

SciLifeLab Serve app template

- Can provide a ready-to-use project structure, best practices, and instructions that AI coding assistants can follow.
- Supports popular frameworks: Streamlit, Gradio, Dash, Shiny (R), and custom Python apps.
- Publishing on SciLifeLab Serve is optional.
  - This template and suggested prompts are useful for web app development regardless of where the application is hosted.

Template: https://github.com/ScilifelabDataCentre/serve-app-template



Two ways to start with the SciLifeLab Serve template

Starting point 1: “I only have an idea”

- Describe the research question, input data, and expected output.
- Specify the scientific methods, calculations, and model choices.
- Explain any assumptions, constraints, and validation criteria.
- Provide relevant datasets, references, or existing scientific specifications.

“‘I only have an idea’ means the scientific task is well-defined, but the code is not yet written. AI must not decide the research methodology.”

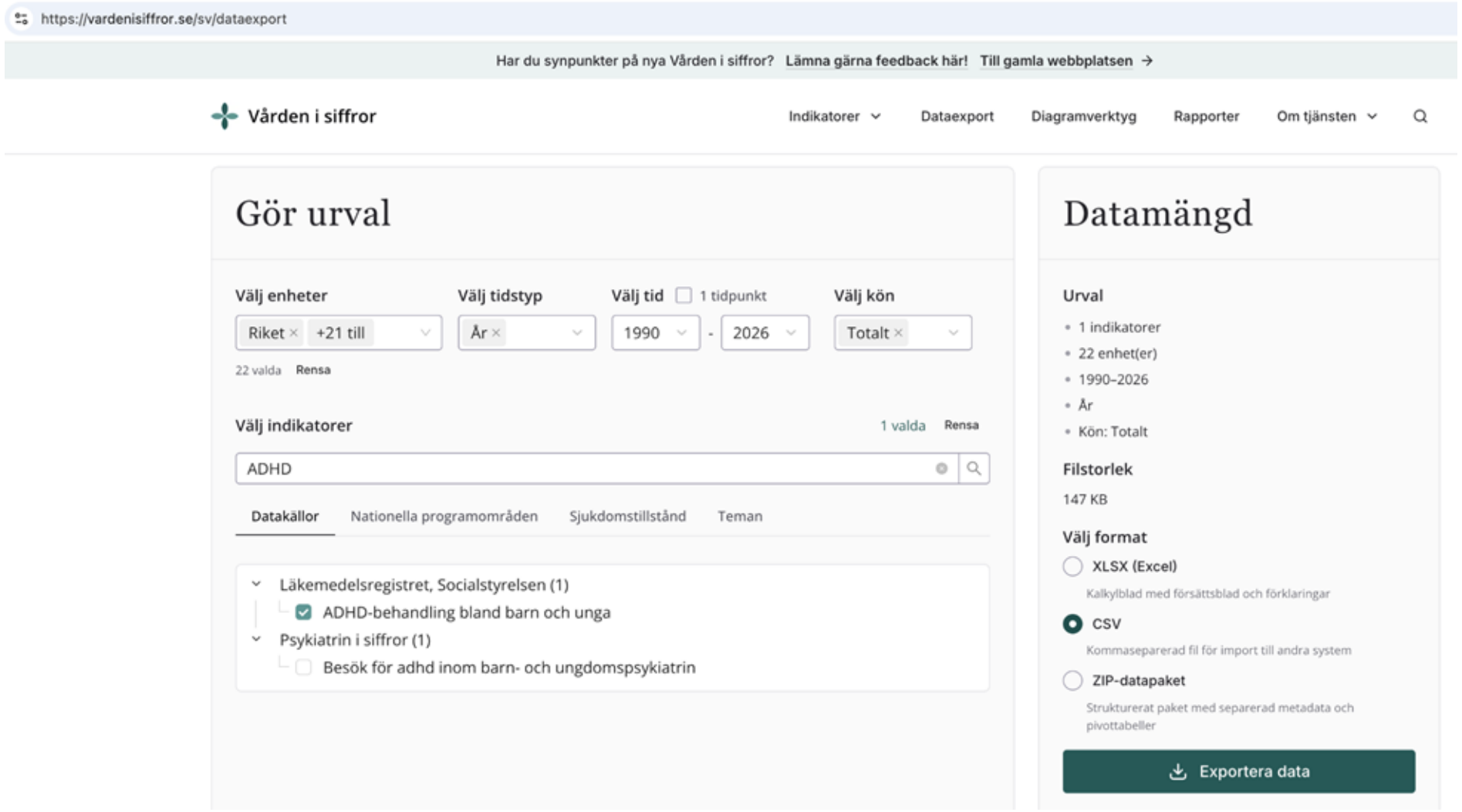
Two ways to start with the SciLifeLab Serve template

Starting point 2: “I already have code”

- Provide your existing code, analysis function, or repository.
- Ask the AI to preserve the scientific logic and make it SciLifeLab Serve-ready.

“Use the template's current README prompts. Check the generated files yourself.”

Public statistics on ADHD medication treatment in Sweden



Our example dataset: ADHD treatment in Sweden

Source: Läkemedelsregistret (National Prescribed Drug Register), Socialstyrelsen, via Vården i siffror.

Indicator: ADHD-behandling bland barn och unga.

Population: Children and young people aged 0–17 years.

Measure: Number receiving ADHD medication treatment per 1,000 inhabitants.

Coverage: 2012–2025, covering 21 Swedish regions and the national total (Riket).

Dataset: 308 observations (22 geographical areas × 14 years), using aggregate, publicly available data.

Our example dataset: ADHD treatment in Sweden

- Prepared dataset: adhd_treatment_sweden_workshop_2012_2025.csv

- Columns:
  - year: Reporting year
  - area: Region name or Riket
  - area_type: Region or National
  - treated_per_1000: ADHD medication treatment rate per 1,000 inhabitants

Streamlit App: ADHD Treatment Explorer

Question: How have ADHD medication treatment rates changed over time and across Swedish regions?

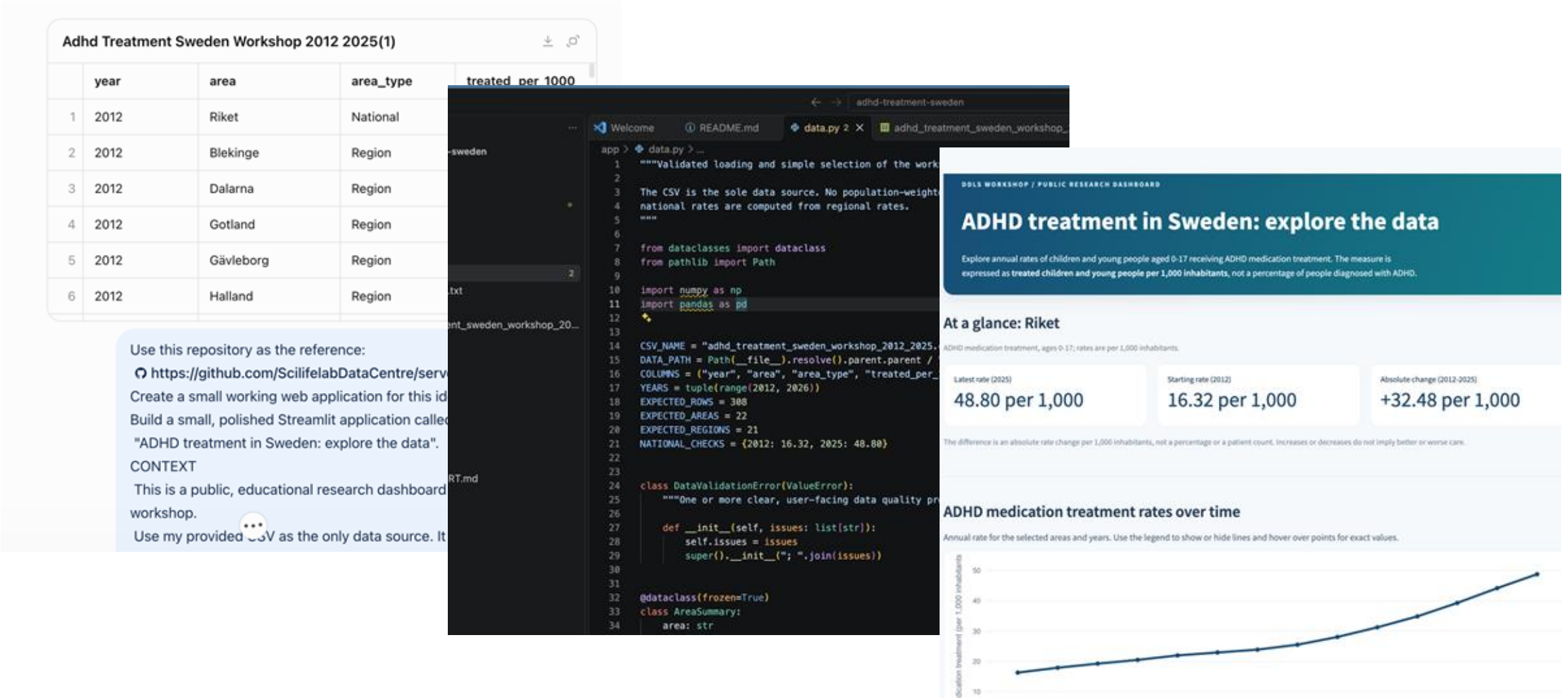
Data: Annual treatment rates from 2012–2025 across 21 regions and Riket.

Inputs: Region selection and year-range filters.

Outputs: Interactive trend charts, regional comparisons, summary statistics, and a downloadable data table.

Primary purpose: Visualize and explore existing research data through an interactive dashboard.

Streamlit App Demo: ADHD Treatment Explorer



SciLifeLab

Part 3: Publishing web apps on SciLifeLab Serve

From working app to container image

- The app and its dependencies live in a project repository.
- Test it locally and verify scientific outputs.
- Package the project as **one Docker image**, following the template instructions.
- Build and test the image locally **or** use the template's GitHub Actions workflow.
- Publish the image to an accessible container registry.
- Keep the **full image reference** and the **application port** for submission.

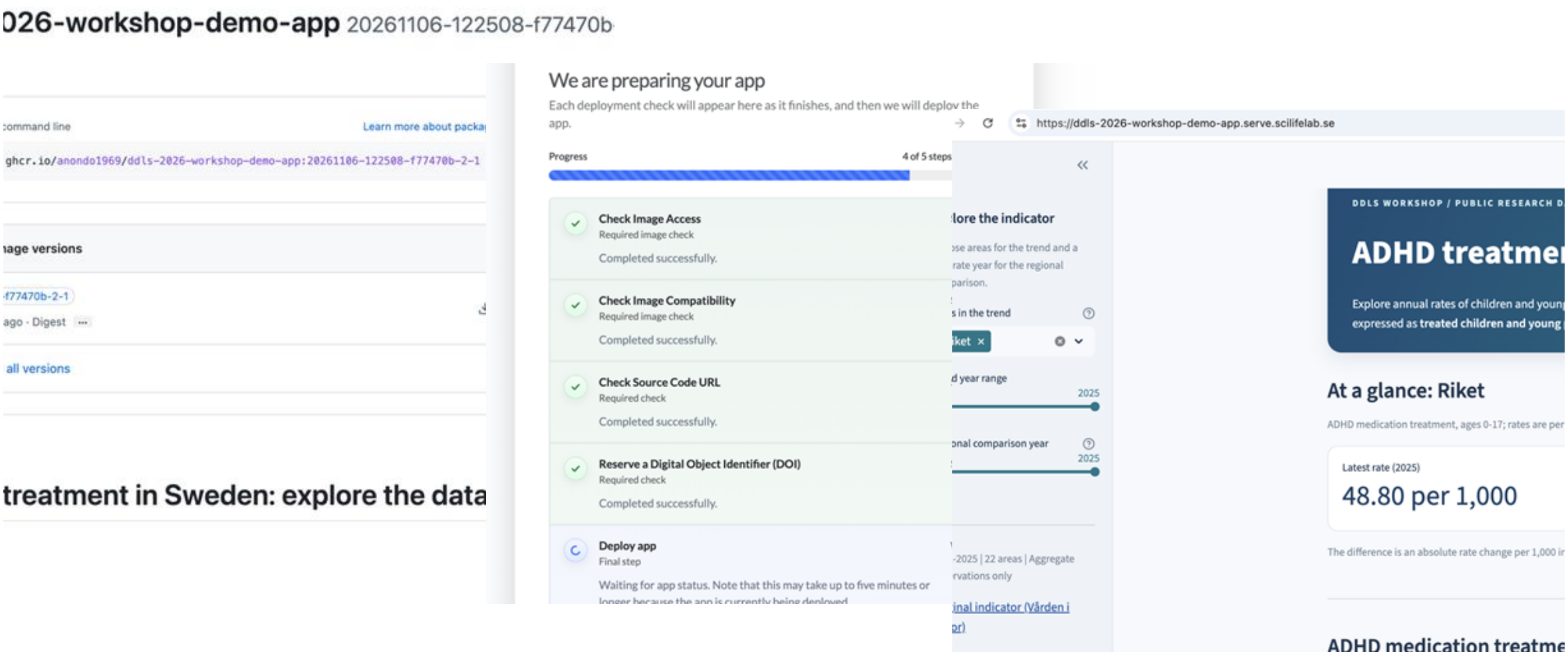
“App files → Docker image → container registry → SciLifeLab Serve”

Publishing an app on SciLifeLab Serve

- Sign in to **SciLifeLab Serve**.
- Create or open a project.
- Use only appropriate **public** data.
- Do not upload **patient-level records or secrets**.
- Read **documentation** for the exact workflow if needed.
- Fill out the app information accurately.
- Provide the **public container-image reference** and **app port**.
- Link the source-code repository and supply appropriate contributors/metadata.
- Submit and test the deployed app.

SciLifeLab Serve: https://serve.scilifelab.se/

Demo: Publishing an app on SciLifeLab Serve





Hands-on session

Hands-on session

Who here brought their own data/code/project to work with?

For everyone else:

Option 1:
Dataset from the National Board of Health and Welfare (Socialstyrelsen) on the number of patients and filled prescriptions for Semaglutid in Sweden between 2016 and 2025, broken down by age, sex, and region. Create an interactive dashboard.

Option 2:
Machine learning model for tree crown detection from Weinstein et al, Remote Sensing 2019. https://doi.org/10.3390/rs11111309. Create an app to make predictions on new data.

Hands-on session



https://github.com/ScilifelabDataCentre/ddls-2026-workshop-materials

Hands-on: build, check, and explain

0–5 min, Pick an option: visualization (1) or analysis (2) or your own

5–15 min, Write a prompt: specify the data definition, interactions, expected output, and checks.

15–25 min, Build and run: make a minimal version, fix errors, and avoid unnecessary features.

25–35 min, Verify

- visualization: compare a plotted value with the original data; test empty filters.
- analysis: independently check one prediction calculation

35–40 min, Wrap up and share (optional): Prepare your web app for sharing through SciLifeLab Serve and add your app with visibility *Project/Private* to test it.

SciLifeLab

Part 4: Funder and institutional requirements

Web apps as research output

- Web apps produced by researchers are research output
  - just like traditional outputs like journal article, conference paper, book chapter and others like dataset, analysis code, etc.



Web apps as research output

- Web apps produced by researchers are research output
  - just like traditional outputs like journal article, conference paper, book chapter and others like dataset, analysis code, etc.
- Relevant guidelines from funders and institutions:
  - Ethical approval [1,2]
  - Open access publication [3,4]
  - Open data [4,5]
  - Open analysis workflows and code [6]
  - Publication of research output according to FAIR principles [1-6]

1. Swedish Research Council, *Conducting ethical research*: https://www.vr.se/english/applying-for-funding/requirements-terms-and-conditions/conducting-ethical-research.html
2. ERC, *Ethics guidance*: https://erc.europa.eu/manage-your-project/ethics-guidance
3. Swedish Research Council, *Publishing your research open access*: https://www.vr.se/english/mandates/open-science/open-access-to-scientific-publications/guidelines-for-publishing-with-open-access.html
4. ERC, *Open Science* https://erc.europa.eu/manage-your-project/open-science
5. Swedish Research Council, *Open access to research data* https://www.vr.se/english/mandates/open-science/open-access-to-research-data/vision-and-guiding-principles.html
6. National guidelines for promoting open science in Sweden https://www.kb.se/samverkan-och-utveckling/nytt-fran-kb/nyheter-samverkan-och-utveckling/2024-01-15-national-guidelines-for-promoting-open-science-in-sweden.html

FAIR principles

Set of principles originally written for research data[1] but since expanded to other research output[2,3].



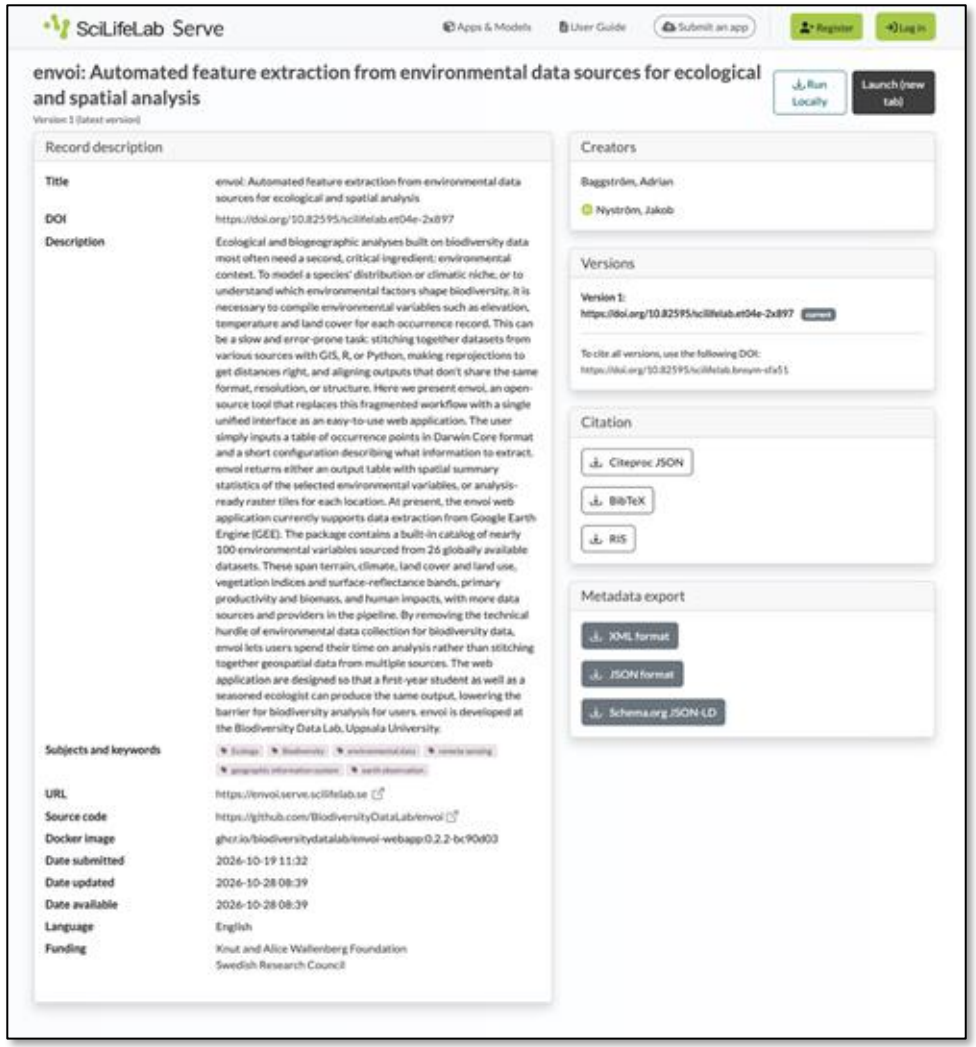
Source: Modified illustration by Scriberia for *The Turing Way* project. CC-BY 4.0 licence. DOI: 10.5281/zenodo.3332807

1. Wilkinson, M., Dumontier, M., Aalbersberg, I. *et al.* The FAIR Guiding Principles for scientific data management and stewardship. *Sci Data* **3**, 160018 (2016). https://doi.org/10.1038/sdata.2016.18
2. Barker, M., Chue Hong, N.P., Katz, D.S. *et al.* Introducing the FAIR Principles for research software. *Sci Data* **9**, 622 (2022). https://doi.org/10.1038/s41597-022-01710-x
3. Patel, B., Soundarajan, S., Ménager, H. *et al.* Making Biomedical Research Software FAIR: Actionable Step-by-step Guidelines with a User-support Tool. *Sci Data* **10**, 557 (2023). https://doi.org/10.1038/s41597-023-02463-x

FAIR when it comes to web apps

- Publish packaged web app according to FAIR requirements
- Publish code according to FAIR requirements
- Publish data according to FAIR requirements

Meeting FAIR requirements: packaged web app



- For all Public applications on SciLifeLab Serve we collect and store metadata and provide a DOI.
- You can refer to a specific version DOI or general DOI for your app.
- Solves:
  - Linking between dataset, code, and running app.
  - Identification of persons, organisations, etc.
  - Versioning.

- Alternative: Deposit the Docker image with your app on a data repository (e.g. Zenodo)

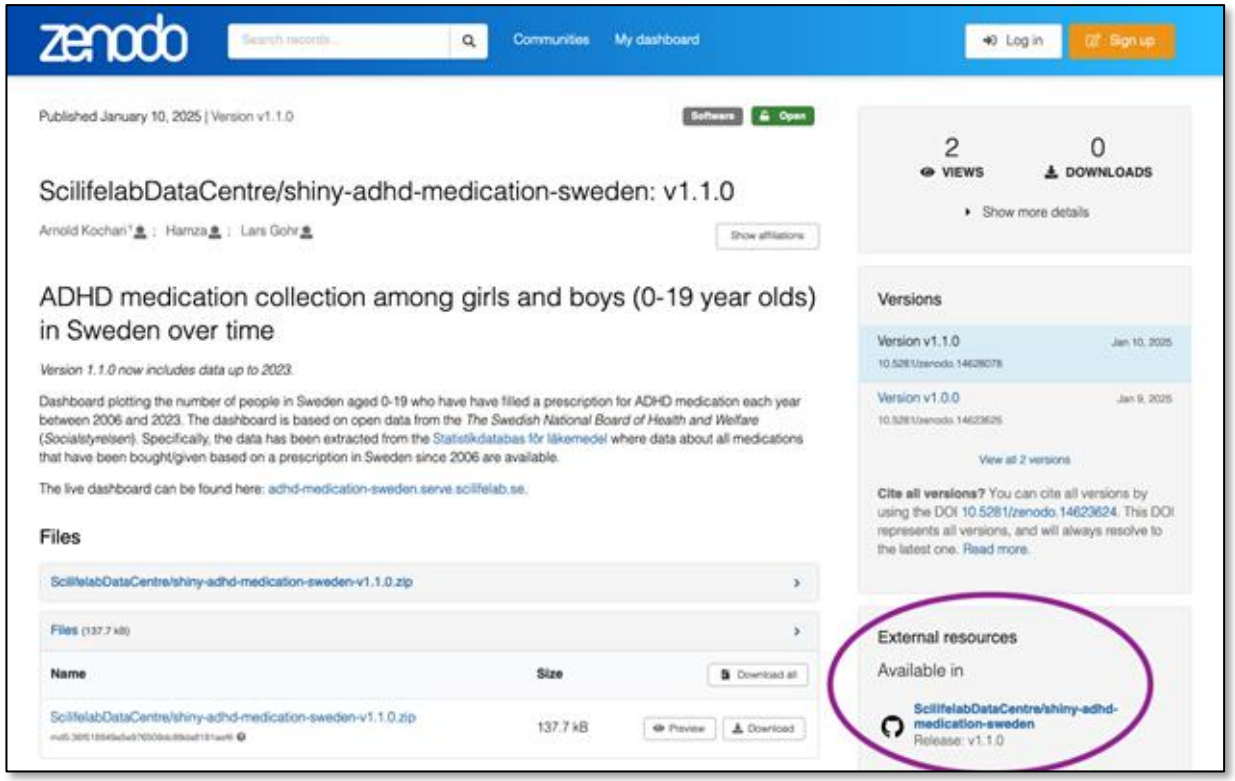
Source: Screenshot of the details page for the app *envoi*: https://serve.scilifelab.se/records/et04e-2x897/

Meeting FAIR requirements: code

In principle to get there 80% of the way:
Deposit a copy of your code to a repository that meets FAIR requirements.

In practice:

- GitHub or similar is great
- Though GitHub is not sufficient for FAIR
  - not an archival repository
  - does not provide a persistent identifier
- Zenodo.org has a great integration with GitHub, you can get a DOI for each release (Figshare offers similar functionality).
- Provide the DOI or another persistent identifier when referring to the code.



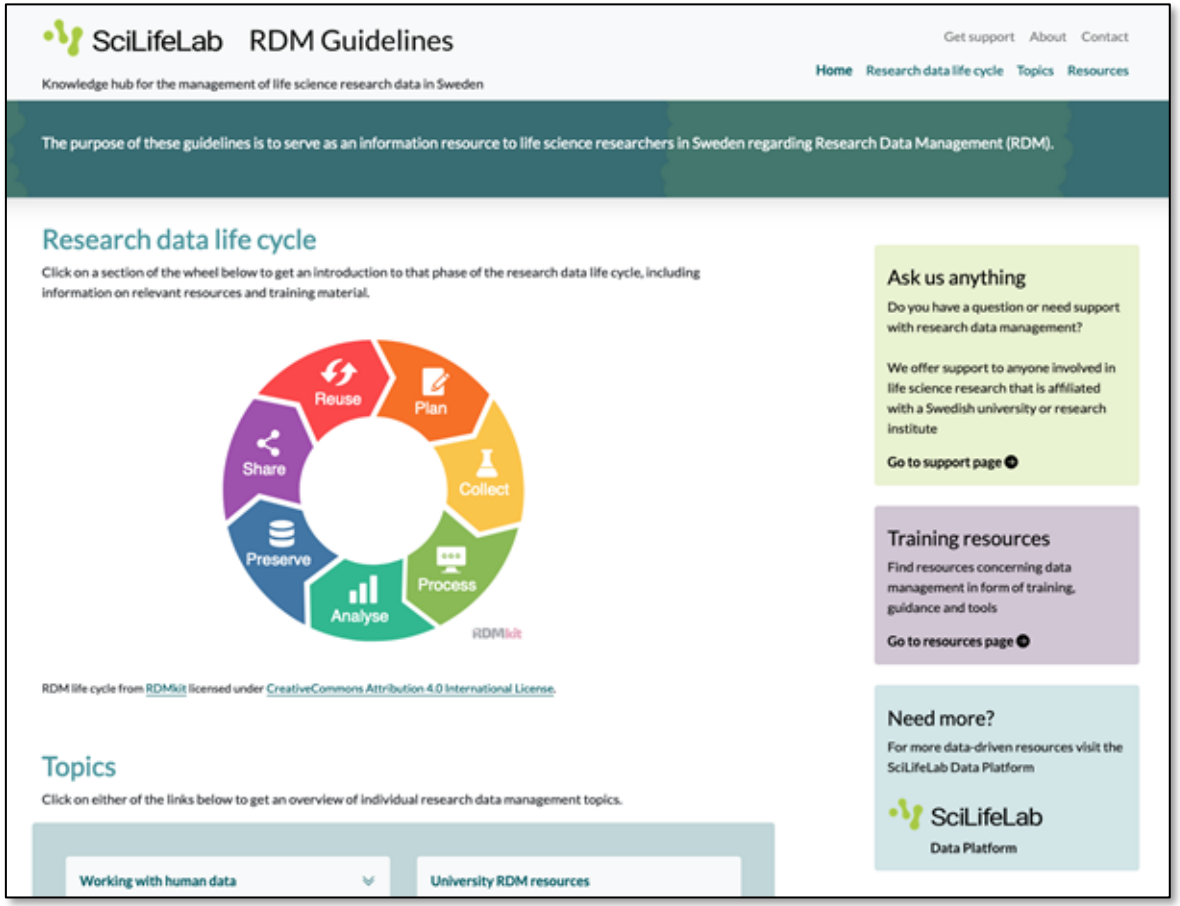
Source: Screenshot of the Zenodo landing page for DOI 10.5281/zenodo.14623624

Meeting FAIR requirements: data

In principle, to get there 80% of the way:
Deposit a copy of your data to a repository that meets FAIR requirements.

In practice:

- Deposit the data even if you already make it available through your app.
- *OR* combine data and web app code in a single entry in a general repository, e.g. Zenodo, Figshare.
- Provide DOI to your data inside your web app.
- Ask for advice from your university data management team or from SciLifeLab/NBIS, available to everyone in Sweden: https://data-guidelines.scilifelab.se/



Source: Screenshot of https://data-guidelines.scilifelab.se

SciLifeLab

Q&A

Thank you!

